

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..33

FLUTTER WEB APPS

 Copy as Markdown

Flutter web apps

Serverpod can serve your Flutter web application directly, allowing you to host both your API and web frontend from the same server. The `FlutterRoute` class handles the specifics: serving the build output, SPA-style fallback to `index.html` for client-side routing, and cache headers that make redeployments take effect right away.

New fullstack projects come with this already set up. The scaffolded `server.dart` serves your Flutter app at the root URL when a build exists in `web/app`, and shows a build-instructions page when it does not.

Basic setup

Use `FlutterRoute` to serve your Flutter web build:

```
pod.webServer.addRoute(  
  FlutterRoute(Directory('web/app')),  
);
```

INFO

The route path defaults to `'/'` (root). See [Serving from a sub-path](#) to mount the app at a different location.

This configuration:

- Serves all static files from the Flutter build
- Falls back to `index.html` for client-side routing
- Serves every file with revalidation-based cache headers, so users get new versions on the next deploy



Building Flutter for web

New fullstack projects ship a build task. Run it from the server directory:

```
$ serverpod run flutter_build
```

It runs the underlying Flutter build and places the output where the server serves it:

```
cd my_project_flutter  
flutter build web --base-href / --output ../my_project_server/web/app
```

If you mount the app at a sub-path such as `/app`, pass that path as `--base-href /app/` so the app's own asset URLs resolve correctly.

The build output ends up next to your server code:

```
my_project/  
├── my_project_server/  
│   ├── lib/  
│   │   └── server.dart  
│   └── web/  
│       └── app/                # Flutter web build output  
│           ├── index.html  
│           ├── main.dart.js  
│           ├── flutter.js  
│           └── ...  
└── my_project_flutter/        # Build source
```

Cache control

By default, `FlutterRoute` serves every file with `Cache-Control: no-cache, private`. The browser keeps a local copy but revalidates it on each load using `ETag` headers, and unchanged files answer with an empty `304 Not Modified`. The cost is one small round-trip per asset rather than a re-download, and the payoff is that a redeploy takes effect on the next page load with no stale-version tricks needed.

This design fits Flutter's build output, which uses fixed file names (such as `main.dart.js`) without content hashing, so longer-lived caching would risk serving a stale app.

Custom cache control

Override the default behavior using `cacheControlFactory`:

```
pod.webServer.addRoute(  
  FlutterRoute(  
    Directory('web/app'),
```

```
cacheControlFactory: StaticRoute.public(
  maxAge: const Duration(hours: 1),
),
),
);
```

⚠ WARNING

A custom factory applies to all files served from the directory. The fallback-served `index.html` (a client-side route like `/dashboard`) always gets no-cache headers, but a direct request to `/index.html` follows your factory, so exclude `index.html` from long-lived caching in a custom strategy.

To set the header per environment instead of in code, set

`SERVERPOD_WEB_SERVER_FLUTTER_CACHE_CONTROL` to the `Cache-Control` value the route should send:

```
SERVERPOD_WEB_SERVER_FLUTTER_CACHE_CONTROL="public, max-age=3600"
```

The environment variable applies to every `FlutterRoute` that does not pass a `cacheControlFactory` of its own, and the same warning applies to it.

See [Static files](#) for more on cache control.

Cache busting

For long-lived caching without stale versions, enable cache-busted URLs:

```
final webDir = Directory('web/app');

final cacheBustingConfig = CacheBustingConfig(
  mountPrefix: '/',
  fileSystemRoot: webDir,
);

pod.webServer.addRoute(
  FlutterRoute(
    webDir,
    cacheBustingConfig: cacheBustingConfig,
    cacheControlFactory: StaticRoute.publicImmutable(
      maxAge: const Duration(days: 365),
    ),
  ),
);
```

See [Static files](#) for more on cache busting.

WASM builds

Flutter can compile to WebAssembly for improved performance, with multi-threaded rendering that requires `SharedArrayBuffer`. Browsers only enable `SharedArrayBuffer` on pages with cross-origin isolation headers:

- `Cross-Origin-Opener-Policy: same-origin`
- `Cross-Origin-Embedder-Policy: require-corp`

The `enableWasmHeaders` parameter controls whether `FlutterRoute` sends them. The constructor defaults it to true, but generated projects pass `enableWasmHeaders: false` because the plain JavaScript build is the scaffolded default and cross-origin isolation has side effects: it blocks embedding cross-origin resources, such as images or iframes from other domains, unless they opt in.

To serve a WASM build:

1. Build with the WASM target:

```
cd my_project_flutter
flutter build web --wasm --base-href / --output ../my_project_server/web/app
```

2. Turn the headers on:

```
pod.webServer.addRoute(  
  FlutterRoute(Directory('web/app'), enableWasmHeaders: true),  
);
```

INFO

WASM builds automatically fall back to JavaScript in browsers that don't support WebAssembly Garbage Collection (WasmGC). Your app works everywhere while taking advantage of WASM performance where available.

Using WasmHeadersMiddleware directly

If you're using `SpaRoute` or custom routes instead of `FlutterRoute`, add the headers with `WasmHeadersMiddleware`:

```
pod.webServer.addMiddleware(const WasmHeadersMiddleware().call, '/');  
  
pod.webServer.addRoute(  
  SpaRoute(  
    Directory('web/app'),  
    fallback: File('web/app/index.html'),
```

```
),  
  '/',  
);
```

Complete example

Here's a `server.dart` excerpt serving a Flutter web app, following the pattern generated projects use:

```
import 'dart:io';  
  
import 'src/generated/serverpod.dart';  
  
void run(List<String> args) async {  
  final pod = Serverpod(args);  
  
  final flutterAppDir = Directory('web/app');  
  
  if (flutterAppDir.existsSync()) {  
    pod.webServer.addRoute(FlutterRoute(flutterAppDir, enableWasmHeaders: false));  
  } else {  
    // No build yet: show instructions instead of a 404.  
    pod.webServer.addRoute(  
      StaticRoute.file(File('web/pages/build_flutter_app.html')),  
      '/*',  
    );  
  }  
  
  await pod.start();  
}
```

With this configuration, your Flutter web app is served at the root URL of your web server (typically `http://localhost:8082` in development).